AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

# DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

## Algorytm drzewa BST
## (binarne drzewo poszukiwań)

Autor:
Damian Kiełbasa
Patryk Gargula

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

# Spis treści

**1. Ogólne określenie wymagań** **4**

1.1. Cel projektu . . . . . 4
1.2. Wymagania funkcjonalne . . . . . 4
1.3. Założenia i ograniczenia . . . . . 4
1.4. Wymagania dotyczące pracy zespołowej i kontroli wersji . . . . . 5

**2. Analiza problemu** **7**

2.1. Drzewo BST i jego zastosowania . . . . . 7
2.2. Dodawanie elementu . . . . . 8
2.3. Usuwanie elementu . . . . . 8
2.4. Przechodzenie drzewa . . . . . 9
2.5. Wyszukiwanie drogi do elementu . . . . . 9
2.6. Złożoność operacji . . . . . 9
2.7. Zapis drzewa do plików . . . . . 10
2.7.1. Format tekstowy . . . . . 10
2.7.2. Format binarny . . . . . 11
2.8. System Git i serwis GitHub . . . . . 11
2.9. Konflikty scalania . . . . . 12
2.10. Struktura repozytorium na GitHubie . . . . . 12
2.11. Pull request i przegląd zmian . . . . . 13
2.12. Inne możliwości serwisu GitHub . . . . . 13

**3. Projektowanie** **15**

3.1. Narzędzia i środowisko . . . . . 15
3.2. Struktura projektu . . . . . 15
3.3. Model klas . . . . . 15
3.4. Interfejs użytkownika . . . . . 16
3.5. Organizacja pracy zespołowej . . . . . 16
3.6. Sposób używania systemu Git . . . . . 17
3.7. Konfiguracja repozytorium i współpracy . . . . . 19
3.8. Przebieg pracy z pull requestem . . . . . 19

3.9. Najczęściej używane polecenia Git . . . . . 20

**4. Implementacja** **21**

4.1. Struktura Node i deklaracja klasy BST . . . . . 21
4.2. Konstruktor i destruktor . . . . . 22
4.3. Dodawanie elementu . . . . . 22
4.4. Usuwanie elementu . . . . . 23
4.5. Usuwanie całego drzewa . . . . . 24
4.6. Wyszukiwanie drogi do elementu . . . . . 25
4.7. Wyświetlanie drzewa . . . . . 25
4.8. Zapis drzewa do pliku tekstowego . . . . . 26
4.9. Klasa FileManager . . . . . 27
4.9.1. Zapis binarny . . . . . 28
4.9.2. Odczyt binarny . . . . . 28
4.9.3. Odczyt z pliku tekstowego . . . . . 29
4.10. Program główny . . . . . 30
4.11. Przykładowy przebieg programu . . . . . 32
4.12. Historia projektu w repozytorium . . . . . 32
4.13. Podsumowanie aktywności w repozytorium . . . . . 33
4.14. Konflikty scalania w projekcie . . . . . 34

**5. Wnioski** **36**

**Literatura** **38**

**Spis rysunków** **39**

**Spis tabel** **40**

**Spis listingów** **41**

# 1. Ogólne określenie wymagań

## 1.1. Cel projektu

Celem projektu jest napisanie w języku C++ programu realizującego **binarne drzewo poszukiwań** (ang. *Binary Search Tree*, BST) alokowane dynamicznie na stercie. Program ma umożliwiać dodawanie i usuwanie elementów, usunięcie całego drzewa, wyszukiwanie drogi od korzenia do wskazanego elementu oraz wyświetlanie drzewa w trzech porządkach przechodzenia: *preorder*, *inorder* i *postorder*[1]. Dodatkowo drzewo ma być możliwe do zapisania i odczytania z pliku, zarówno tekstowego, jak i binarnego.

Obsługa programu odbywa się z poziomu menu tekstowego uruchamianego w konsoli. Za operacje na plikach odpowiada osobna klasa, dzięki czemu logika samego drzewa jest oddzielona od obsługi zapisu i odczytu danych.

Przewidywane rezultaty projektu to:

- działający program z kompletem operacji na drzewie BST,
- możliwość zapisu i odczytu drzewa w formacie tekstowym i binarnym,
- interaktywne menu umożliwiające wywołanie każdej z operacji,
- niniejsza dokumentacja projektowa przygotowana w LaTeXu,
- repozytorium na GitHubie z historią zmian pokazującą podział prac w zespole i kolejne etapy projektu.

## 1.2. Wymagania funkcjonalne

Program udostępnia operacje wymienione w tabeli 1.1. W tabeli podano również numer opcji w menu oraz nazwy metod, którymi zostały zrealizowane poszczególne wymagania.

## 1.3. Założenia i ograniczenia

Przy projektowaniu programu przyjęto następujące założenia:

- drzewo przechowuje wartości całkowite typu `int`,

[1] Podstawy drzew poszukiwań: [1].

**Tab. 1.1.** Wymagania funkcjonalne i ich realizacja

| Lp. | Wymaganie | Opcja menu | Metoda |
|---|---|---|---|
| 1 | Dodaj element do drzewa | 1 | `BST::insert` |
| 2 | Usuń wskazany element z drzewa | 2 | `BST::remove` |
| 3 | Usuń całe drzewo | 3 | `BST::clear` |
| 4 | Wyszukaj drogę do elementu | 4 | `BST::searchPath` |
| 5 | Wyświetl drzewo (pre-, in-, postorder) | 5 | `BST::display` |
| 6 | Zapisz drzewo do pliku tekstowego | 6 | `BST::saveToTextFile` |
| 7 | Zapisz drzewo do pliku binarnego | 7 | `FileManager::saveBinary` |
| 8 | Wczytaj drzewo z pliku binarnego | 8 | `FileManager::loadBinary` |
| 9 | Wczytaj liczby z pliku tekstowego | 9 | `FileManager::loadFromText` |

- w drzewie nie występują duplikaty – próba dodania istniejącej już wartości nie zmienia drzewa,
- usuwanie nieistniejącego elementu nie zmienia drzewa i nie powoduje błędu,
- w pliku binarnym wartość `-1` pełni rolę znacznika pustego węzła, więc liczba $-1$ nie może być poprawnie zapisana w tym formacie (rozdział 2.7.2),
- program nie zawiera walidacji danych wprowadzanych z klawiatury.

## 1.4. Wymagania dotyczące pracy zespołowej i kontroli wersji

Projekt był realizowany w dwuosobowym zespole, dlatego w trakcie pracy przyjęto następujące zasady:

- kod projektu jest przechowywany w repozytorium Git na serwisie GitHub,
- każdy z autorów pracuje na własnej gałęzi, a jego zmiany trafiają do gałęzi głównej przez *pull request*,

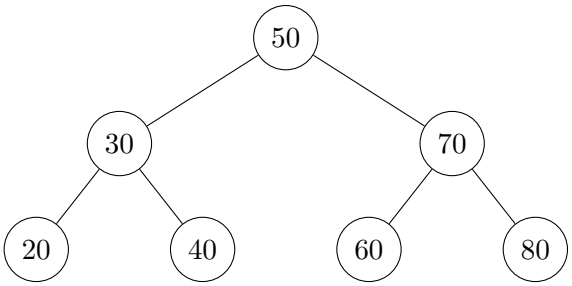
- commity obejmują niewielkie, zakończone zmiany, a ich opisy mówią, co zostało zrobione,
- w repozytorium znajdują się pliki `.gitignore`, `.gitattributes` oraz opis projektu `README`,
- ewentualne konflikty powstające przy scalaniu gałęzi mają być rozwiązywane ręcznie, z zachowaniem zmian obu autorów.

# 2. Analiza problemu

## 2.1. Drzewo BST i jego zastosowania

Binarne drzewo poszukiwań jest dynamiczną strukturą danych złożoną z węzłów. Każdy węzeł przechowuje wartość oraz dwa wskaźniki: na lewego i na prawego potomka. Brak potomka oznacza wartość `nullptr`. Pierwszy węzeł drzewa nazywa się **korzeniem**, a węzeł bez potomków – **liściem**.

Cechą wyróżniającą drzewo BST jest następująca zależność, spełniona dla każdego węzła: wszystkie wartości w jego lewym poddrzewie są **mniejsze** od wartości węzła, a wszystkie wartości w prawym poddrzewie są **większe**. Przykładowe drzewo przedstawia rysunek 2.1.




**Rys. 2.1.** Przykładowe drzewo BST utworzone przez dodanie wartości 50, 30, 70, 20, 40, 60, 80

Dzięki opisanej zależności w drzewie można szybko odnajdywać elementy: w każdym węźle wystarczy porównać szukaną wartość z wartością węzła i wybrać jedno z poddrzew, co odrzuca około połowy pozostałych elementów (o ile drzewo jest zrównoważone). Drzewa poszukiwań stosuje się między innymi w:

- słownikach i zbiorach przechowujących elementy w uporządkowanej kolejności,
- indeksach baz danych (w postaci bardziej zaawansowanych drzew zrównoważonych),
- implementacjach kontenerów typu zbiór i mapa w bibliotekach standardowych,
- algorytmach wymagających szybkiego wstawiania, usuwania i wyszukiwania,
- sortowaniu – przejście *inorder* daje wartości w kolejności rosnącej.

## 2.2. Dodawanie elementu

Dodawanie wartości polega na zejściu od korzenia w dół drzewa. W każdym węźle porównuje się dodawaną wartość z wartością węzła: jeśli jest mniejsza, przechodzi się do lewego poddrzewa, jeśli większa – do prawego. Gdy napotkane zostanie puste miejsce, tworzony jest tam nowy liść. Jeśli wartość jest równa wartości któregoś z węzłów, drzewo pozostaje bez zmian, ponieważ duplikaty nie są przechowywane.

## 2.3. Usuwanie elementu

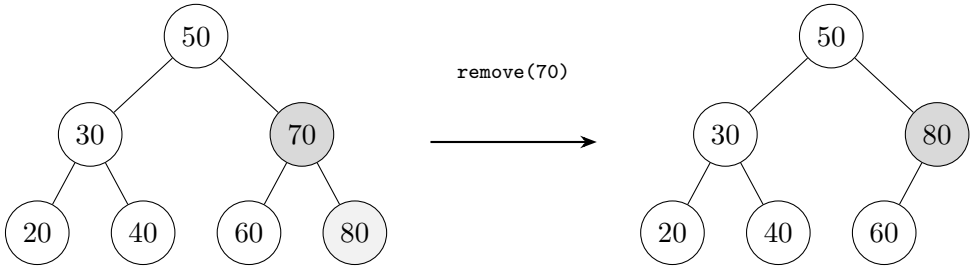
Usuwanie jest najbardziej złożoną operacją na drzewie BST. Po odnalezieniu węzła do usunięcia (w ten sam sposób jak przy dodawaniu) rozróżnia się trzy przypadki:

1. **węzeł jest liściem** – jest po prostu zwalniany, a wskaźnik w jego rodzicu zostaje wyzerowany,

2. **węzeł ma jednego potomka** – węzeł jest zwalniany, a na jego miejsce w rodzicu wstawiany jest jedyny potomek,

3. **węzeł ma dwóch potomków** – wartość węzła zastępowana jest wartością najmniejszego elementu z jego prawego poddrzewa (tzw. następnika w porządku *inorder*), a następnie ten najmniejszy element jest usuwany z prawego poddrzewa.

Zastąpienie wartości następnikiem zachowuje własność drzewa BST, ponieważ następnik jest większy od wszystkich wartości w lewym poddrzewie i nie większy od pozostałych wartości w prawym poddrzewie. Trzeci przypadek zilustrowano na rysunku 2.2 na przykładzie usunięcia wartości 70 z drzewa z rysunku 2.1.




**Rys. 2.2.** Usunięcie węzła z dwoma potomkami: wartość 70 zostaje zastąpiona następnikiem (80)

## 2.4. Przechodzenie drzewa

Wyświetlenie wszystkich elementów drzewa wymaga odwiedzenia każdego węzła. Robi się to rekurencyjnie, a o wyniku decyduje moment, w którym przetwarzana jest wartość bieżącego węzła w stosunku do odwiedzin obu poddrzew (tabela 2.1).

**Tab. 2.1.** Porządki przechodzenia drzewa i wyniki dla drzewa z rys. 2.1

| **Porządek** | **Kolejność odwiedzin** | **Wynik** |
|---|---|---|
| *preorder* | węzeł, lewe, prawe | 50 30 20 40 70 60 80 |
| *inorder* | lewe, węzeł, prawe | 20 30 40 50 60 70 80 |
| *postorder* | lewe, prawe, węzeł | 20 40 30 60 80 70 50 |

Przejście *inorder* zwraca wartości w porządku rosnącym, co jest bezpośrednim skutkiem własności drzewa BST. Przejście *preorder* ma natomiast ważną cechę: dodanie wartości do pustego drzewa w kolejności *preorder* odtwarza drzewo o identycznym kształcie. Z tej własności korzysta zapis drzewa do pliku tekstowego (rozdział 2.7.1). Przejście *postorder* wykorzystano przy usuwaniu całego drzewa, ponieważ potomkowie muszą zostać zwolnieni przed swoim rodzicem.

## 2.5. Wyszukiwanie drogi do elementu

Wyszukiwanie drogi polega na zapamiętaniu kolejnych wartości węzłów odwiedzonych w drodze od korzenia do szukanego elementu. W implementacji używana jest rekurencja z listą odwiedzonych wartości: węzeł jest dopisywany do listy przy wejściu, a usuwany z niej, jeśli żadne z jego poddrzew nie zawiera szukanej wartości. Dla drzewa z rysunku 2.1 droga do wartości 60 to 50, 70, 60.

Warto zauważyć, że zastosowany algorytm przeszukuje oba poddrzewa każdego węzła (najpierw lewe, potem prawe) i nie wykorzystuje porządku wartości w drzewie. Poprawność wyniku jest zachowana, jednak w najgorszym przypadku odwiedzane są wszystkie węzły.

## 2.6. Złożoność operacji

Czas działania operacji dodawania, usuwania i (w klasycznej wersji) wyszukiwania zależy od **wysokości drzewa** $h$, czyli liczby węzłów na najdłuższej drodze od korzenia do liścia. Dla drzewa zrównoważonego $h \approx \log_2 n$, natomiast dla drzewa

zdegenerowanego, powstałego np. przez dodawanie wartości w kolejności rosnącej, drzewo przypomina listę i $h = n$. Zestawienie złożoności zawiera tabela 2.2.

**Tab. 2.2.** Złożoność czasowa operacji na drzewie BST

| **Operacja** | **Złożoność** | **Uzasadnienie** |
|---|---|---|
| `insert` | $O(h)$: średnio $O(\log n)$, najgorzej $O(n)$ | zejście wzdłuż jednej gałęzi |
| `remove` | $O(h)$: średnio $O(\log n)$, najgorzej $O(n)$ | odnalezienie węzła i następnika |
| `searchPath` (w tej implementacji) | $O(n)$ | możliwe przeszukanie obu poddrzew |
| `display`, `saveToTextFile` | $O(n)$ | odwiedzenie każdego węzła |
| `clear` | $O(n)$ | zwolnienie każdego węzła |
| `saveBinary`, `loadBinary` | $O(n)$ | jeden zapis/odczyt na węzeł lub znacznik |
| `loadFromText` | $O(n \cdot h)$ | $n$ wywołań `insert` |

Ponieważ operacje są zapisane rekurencyjnie, głębokość stosu wywołań jest równa wysokości drzewa. Dla bardzo dużych, zdegenerowanych drzew może to prowadzić do przepełnienia stosu.

## 2.7. Zapis drzewa do plików

Program obsługuje dwa formaty zapisu drzewa. Oba korzystają z przejścia *preorder*, jednak różnią się sposobem zapisu i przeznaczeniem.

### 2.7.1. Format tekstowy

Plik tekstowy zawiera wartości węzłów w kolejności *preorder*, oddzielone spacjami. Jest czytelny dla człowieka i można go edytować w dowolnym edytorze tekstu. Odczyt polega na wczytywaniu kolejnych liczb i dodawaniu ich do drzewa metodą `insert`. Zgodnie z własnością przejścia *preorder* odczytane drzewo ma taki sam kształt jak zapisane. Ten sam mechanizm pozwala wczytać do drzewa dowolny plik

tekstowy z liczbami (opcja 9 w menu), niekoniecznie wcześniej zapisany przez program.

### 2.7.2. Format binarny

Plik binarny przechowuje strukturę drzewa wraz z jego kształtem. Każdy węzeł zapisywany jest jako jedna liczba typu `int`, a w miejscu każdego brakującego potomka zapisywany jest znacznik `-1`. Zapis odbywa się w kolejności *preorder*. Dla drzewa z rysunku 2.1 zawartość pliku przedstawia tabela 2.3.

**Tab. 2.3.** Zawartość pliku binarnego dla drzewa z rys. 2.1 (15 liczb typu `int`)

| **Element** | **Zapisane wartości** |
|---|---|
| Węzły (7) | 50, 30, 20, 40, 70, 60, 80 |
| Znaczniki (8) | $-1$ po każdym brakującym potomku |
| Kolejność całości | 50 30 20 $-1$ $-1$ 40 $-1$ $-1$ 70 60 $-1$ $-1$ 80 $-1$ $-1$ |

Znaczniki jednoznacznie określają kształt drzewa, dlatego odczyt odtwarza dokładnie tę samą strukturę bez ponownego wstawiania elementów. Konsekwencją użycia wartości $-1$ jako znacznika jest brak możliwości zapisania w tym formacie liczby $-1$ jako wartości węzła. Przy 4-bajtowym typie `int` plik dla powyższego drzewa zajmuje 60 bajtów.

## 2.8. System Git i serwis GitHub

Git to rozproszony system kontroli wersji[2]. Zapamiętuje kolejne stany projektu w postaci **commitów**, czyli migawek wszystkich plików wraz z opisem zmian, autorem i datą. Dzięki temu każdy stan projektu można porównać z innym lub do niego wrócić. Najważniejsze pojęcia to:

- **repozytorium** – katalog projektu wraz z historią zmian,
- **commit** – zapisany w historii stan projektu,
- **gałąź** (*branch*) – równoległa linia rozwoju projektu, na której można pracować bez wpływu na pozostałe gałęzie,

[2]Opis systemu Git: [4].

- **scalanie** (*merge*) – połączenie zmian z jednej gałęzi z drugą,
- **pull request** – propozycja scalenia gałęzi zgłaszana w serwisie GitHub, umożliwiająca przejrzenie zmian przed ich włączeniem,
- **repozytorium zdalne** (*remote*) – kopia repozytorium na serwerze; zmiany wysyła się poleceniem `git push`, a pobiera `git pull`.

GitHub[3] jest serwisem internetowym hostującym repozytoria Git. Pełni rolę wspólnego, zdalnego repozytorium, dzięki któremu możliwa jest praca kilku osób nad jednym projektem i synchronizacja kodu między komputerami.

## 2.9. Konflikty scalania

Gdy dwie osoby pracują na osobnych gałęziach, Git potrafi automatycznie połączyć ich zmiany, o ile dotyczą one różnych fragmentów plików. **Konflikt scalania** powstaje wtedy, gdy obie gałęzie zmieniły te same lub sąsiadujące linie pliku i Git nie może samodzielnie zdecydować, którą wersję zachować. Dotyczy to w szczególności plików, do których każdy z autorów dopisuje własny fragment, np. listy deklaracji w klasie lub menu programu.

W takiej sytuacji Git przerywa scalanie, a w plikach konfliktowych umieszcza znaczniki `<<<<<<<`, `=======` i `>>>>>>>`, między którymi znajdują się obie wersje fragmentu. Rozwiązanie konfliktu składa się z kilku kroków:

1. sprawdzenie, których plików dotyczy konflikt (`git status`),
2. ręczne edytowanie plików: wybór lub połączenie obu wersji i usunięcie znaczników,
3. skompilowanie programu i sprawdzenie, czy działa,
4. oznaczenie plików jako rozwiązanych (`git add`),
5. zapisanie wyniku scalenia w historii (`git commit`).

## 2.10. Struktura repozytorium na GitHubie

Repozytorium projektu w serwisie GitHub zawiera nie tylko kod programu, ale także pliki pomocnicze, które porządkują pracę z Gitem (tabela 2.4). Pliki `.gitignore` i `.gitattributes` dodano na samym początku pracy (commit `d251959`), a plik `README` – pod koniec pierwszego etapu (commit `24f9f25`).

[3]Dokumentacja serwisu: [5].

**Tab. 2.4.** Elementy repozytorium projektu

| **Element** | **Przeznaczenie** |
|---|---|
| `.gitignore` | lista plików i katalogów pomijanych przez Git, np. plików wynikowych kompilacji i plików tymczasowych środowiska Visual Studio |
| `.gitattributes` | ustawienia sposobu traktowania plików przez Git, m.in. końców linii |
| `README` | opis projektu wyświetlany na stronie głównej repozytorium |
| pliki `.h`, `.cpp` | kod źródłowy programu |
| gałęzie (*branches*) | równoległe linie rozwoju projektu, osobne dla każdego autora i etapu |
| historia commitów | pełny zapis zmian z autorem, datą i opisem |

## 2.11. Pull request i przegląd zmian

**Pull request** to mechanizm serwisu GitHub służący do włączania zmian z jednej gałęzi do drugiej. W odróżnieniu od polecenia `git merge` wykonanego lokalnie pozwala on najpierw przejrzeć zmiany: GitHub wyświetla listę zmienionych plików i linii oraz informuje, czy gałęzie można scalić automatycznie, czy konieczne jest rozwiązanie konfliktów. Do pull requesta można dodawać komentarze, dzięki czemu autor i druga osoba z zespołu mogą omówić zmiany przed ich scaleniem.

Po scaleniu w historii gałęzi głównej powstaje commit o opisie *Merge pull request #n from …*, a numer #n jednoznacznie identyfikuje pull request w repozytorium. Scalenia wykonane w serwisie GitHub są podpisywane przez serwis i oznaczane jako *Verified*, co widać w historii projektu.

## 2.12. Inne możliwości serwisu GitHub

Poza funkcjami wykorzystanymi w projekcie (repozytorium, gałęzie, pull requesty, historia commitów) GitHub oferuje m.in.:

- **Issues** – zgłaszanie błędów i zadań do wykonania,
- **Projects** – tablice zadań w stylu kanban,
- **Actions** – automatyczne uruchamianie testów i kompilacji po każdej zmianie,

- **Releases** – publikowanie gotowych wersji programu,
- **Wiki** – dodatkową dokumentację projektu,
- **Insights** – statystyki aktywności, w tym wykres gałęzi (*Network*) i listę współautorów (*Contributors*).

W większych zespołach funkcje te pozwalają zautomatyzować i uporządkować pracę, w projekcie o niewielkim zakresie wystarczyły podstawowe mechanizmy.

# 3. Projektowanie

## 3.1. Narzędzia i środowisko

Program napisano w języku C++[4]. Wymagany jest kompilator obsługujący standard C++11 (w kodzie używane jest słowo kluczowe `nullptr`). Do działania programu wystarcza biblioteka standardowa: nagłówki `<iostream>`, `<vector>`, `<fstream>` i `<string>`; nie są używane żadne dodatkowe biblioteki.

Narzędzia wykorzystane w projekcie:

- **Visual Studio 2022** – środowisko programistyczne i kompilator,
- **Git** i **GitHub** – kontrola wersji, zdalne repozytorium i praca zespołowa,
- **Overleaf** – przygotowanie niniejszej dokumentacji w LaTeXu.

## 3.2. Struktura projektu

Projekt składa się z pięciu plików źródłowych (tabela 3.1). Logika drzewa, obsługa plików binarnych i interfejs użytkownika zostały rozdzielone między osobne jednostki.

**Tab. 3.1.** Pliki projektu

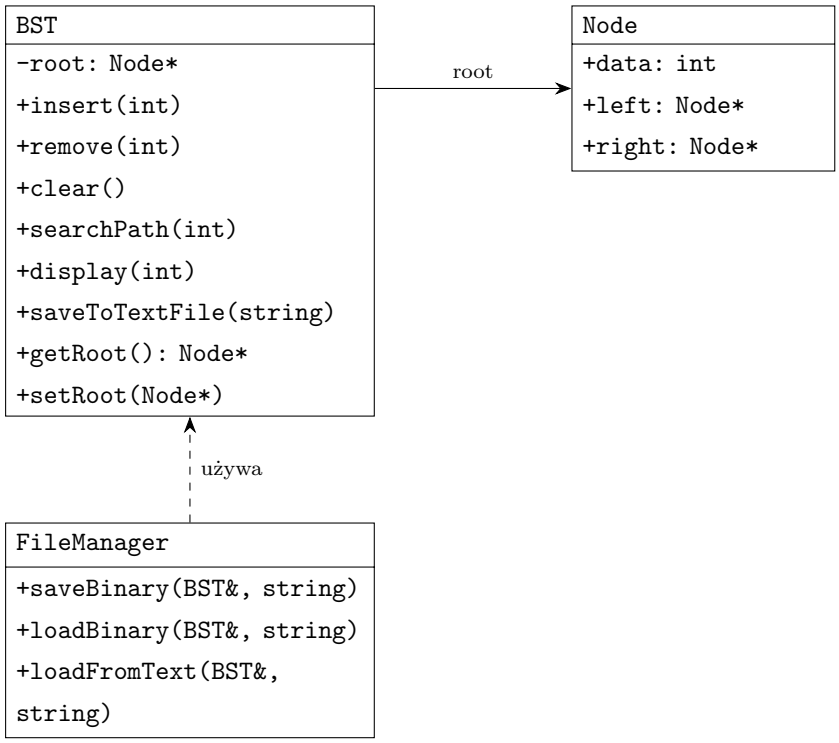
| **Plik** | **Zawartość** |
|---|---|
| `BST.h` | struktura `Node` i deklaracja klasy `BST` |
| `BST.cpp` | implementacja metod klasy `BST` |
| `FileManager.h` | deklaracja klasy `FileManager` |
| `FileManager.cpp` | implementacja metod klasy `FileManager` |
| `main.cpp` | menu użytkownika i pętla główna programu |

## 3.3. Model klas

W programie występują trzy elementy: struktura `Node`, klasa `BST` oraz klasa `FileManager`. Ich zależności przedstawia rysunek 3.1. Klasa `BST` przechowuje wskaźnik `root` na korzeń drzewa zbudowanego z obiektów `Node`. Klasa `FileManager` nie

[4]Opis języka: [2], [3].

przechowuje żadnego stanu – operuje na przekazanym jej drzewie, korzystając z metod `getRoot`, `setRoot`, `clear` i `insert`.




**Rys. 3.1.** Model klas programu

## 3.4. Interfejs użytkownika

Program uruchamia pętlę, w której wyświetlane jest menu, a następnie wczytywany jest numer opcji. Pętla kończy się po wybraniu opcji 0. Wybranie numeru spoza listy powoduje ponowne wyświetlenie menu. Pozycje menu opisuje tabela 3.2.

## 3.5. Organizacja pracy zespołowej

Zadania podzielono między dwóch autorów: Damiana (konto `Damian701`) i Patryka (konto `pgarhub`). Podział, ustalony na podstawie historii commitów, przedstawia tabela 3.3.

Prace podzielono na dwa etapy. W każdym z nich autorzy pracowali na osobnych gałęziach, które po zakończeniu zadania były scalane z gałęzią główną przez *pull request* (tabela 3.4).

**Tab. 3.2.** Opcje menu programu

| Opcja | Opis | Dane wprowadzane |
|---|---|---|
| 1 | Dodaj element | wartość |
| 2 | Usuń element z drzewa | wartość |
| 3 | Usuń całe drzewo | – |
| 4 | Szukaj drogi do elementu | wartość |
| 5 | Wyświetl drzewo | metoda: 1 – pre, 2 – in, 3 – post |
| 6 | Zapisz drzewo do pliku TXT | nazwa pliku |
| 7 | Zapisz drzewo binarnie | nazwa pliku |
| 8 | Wczytaj drzewo binarnie | nazwa pliku |
| 9 | Wczytaj liczby z pliku TXT | nazwa pliku |
| 0 | Wyjście | – |

**Tab. 3.3.** Podział zadań w zespole

| Autor | Zakres prac |
|---|---|
| Damian | struktura `Node` i deklaracja klasy `BST`, dodawanie elementów do drzewa, główne menu i test dodawania, czyszczenie drzewa i szukanie drogi, plik `README`, dokumentacja projektowa |
| Patryk | struktura plików projektu, pliki `.gitattributes` i `.gitignore`, wyświetlanie drzewa (deklaracje, implementacja *preorder*, *inorder*, *postorder*, opcja w menu), menedżer plików i obsługa zapisu i odczytu |

## 3.6. Sposób używania systemu Git

Praca nad projektem przebiegała w cyklu: pobranie aktualnej wersji gałęzi głównej (w repozytorium nazywa się `master`), utworzenie własnej gałęzi, zmiana kodu, sprawdzenie, czy program się kompiluje, dodanie plików do poczekalni (`git add`) i zapisanie ich w historii (`git commit`). Po zakończeniu zadania gałąź była wysyłana na GitHub (`git push`), a następnie scalana z gałęzią główną przez *pull request*. Polecenia użyte w tym cyklu, wraz z opisem commitu pochodzącym z historii projektu,

**Tab. 3.4.** Gałęzie i *pull requesty* w projekcie

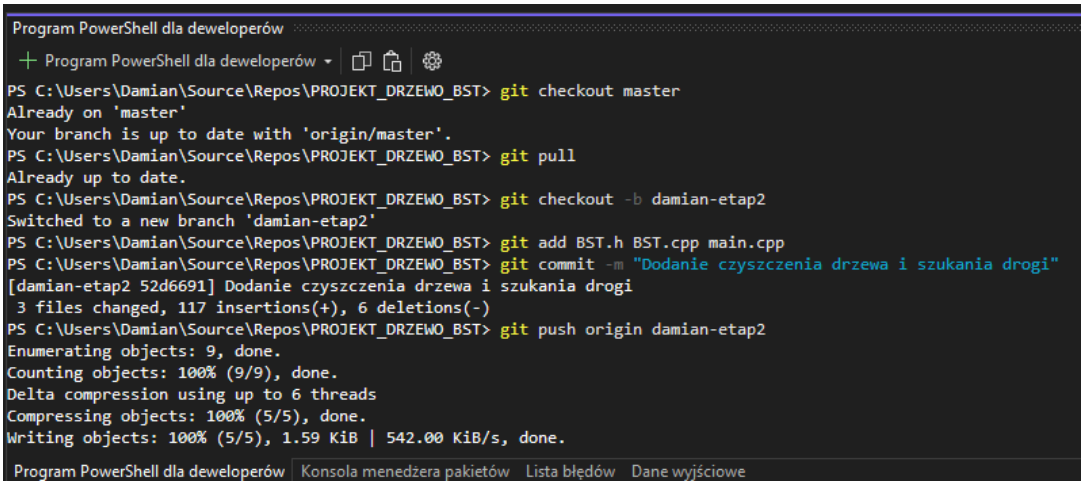
| Gałąź | Autor | Zakres | PR |
|---|---|---|---|
| `galaz-damiana` | Damian | struktura `Node`, deklaracja `BST`, dodawanie, główne menu | #1 |
| `galaz-patryka` | Patryk | wyświetlanie drzewa (deklaracje, implementacja, opcja w menu) | #2 |
| `damian-etap2` | Damian | czyszczenie drzewa i szukanie drogi | #3 |
| `patryk-etap2` | Patryk | menedżer plików, zapis i odczyt | #4 |

przedstawia listing 1, a ich przebieg w terminalu – rysunek 3.2.

```
git checkout master                    # przejscie na galaz glowna
git pull                               # pobranie aktualnej wersji
git checkout -b damian-etap2           # utworzenie wlasnej galezi
git add BST.h BST.cpp main.cpp         # dodanie plikow do
    poczekalni
git commit -m "Dodanie czyszczenia drzewa i szukania drogi"
git push origin damian-etap2           # wyslanie galezi na GitHub
# pull request tworzony i scalany w serwisie GitHub
```

**Listing 1.** Polecenia Git użyte podczas commitowania

Na rysunku 3.2 widać, że commit o skrócie `52d6691` (por. tabela 4.1) objął trzy pliki: `BST.h`, `BST.cpp` i `main.cpp`, w których dodano 117 linii i usunięto 6.




**Rys. 3.2.** Zrzut ekranu terminala: utworzenie gałęzi, commit i wysłanie zmian na GitHub

## 3.7. Konfiguracja repozytorium i współpracy

Repozytorium założono w serwisie GitHub na koncie Patryka (`pgarhub`). Aby Damian mógł wysyłać własne gałęzie bezpośrednio do tego repozytorium, otrzymał do niego dostęp jako współpracownik (ang. *collaborator*). Konfiguracja przebiegała w następujących krokach:

1. założenie repozytorium w serwisie GitHub,
2. dodanie plików `.gitignore` i `.gitattributes`,
3. nadanie dostępu drugiemu autorowi,
4. sklonowanie repozytorium na komputer (`git clone`) i ustawienie danych autora commitów (`git config`),
5. rozpoczęcie pracy na własnych gałęziach.

Polecenia były wydawane w terminalu zintegrowanym ze środowiskiem Visual Studio (rysunek 3.2). Podstawowe polecenia konfiguracji i przeglądania repozytorium przedstawia listing 2.

```
git config --global user.name "Imie Nazwisko"   # dane autora commitow
git config --global user.email "adres@email.pl"
git clone <adres_repozytorium>        # pobranie repozytorium z GitHuba
git branch                            # lista galezi lokalnych
git status                            # stan plikow
git log --oneline                     # skrocona historia commitow
```

**Listing 2.** Podstawowe polecenia konfiguracji i przeglądania repozytorium

## 3.8. Przebieg pracy z pull requestem

Każde zadanie kończyło się scaleniem gałęzi z gałęzią główną przez pull request. Na przykładzie gałęzi `damian-etap2` (pull request #3, scalenie `058ffe8`) procedura wyglądała następująco:

1. wysłanie gałęzi na GitHub poleceniem `git push origin damian-etap2` (listing 1),
2. utworzenie pull requesta w serwisie GitHub z gałęzi `damian-etap2` do gałęzi `master`, z tytułem i opisem zmian,

3. przejrzenie listy zmienionych plików i sprawdzenie, czy GitHub pozwala na automatyczne scalenie,
4. scalenie pull requesta; w historii gałęzi głównej pojawia się commit *Merge pull request #3 from pgarhub/damian-etap2*,
5. powrót lokalnie na gałąź `master` i pobranie scalonych zmian poleceniem `git pull`, aby kolejna gałąź startowała z aktualnej wersji.

W przypadku wszystkich czterech pull requestów (tabela 3.4) GitHub umożliwił scalenie bez rozwiązywania konfliktów.

## 3.9. Najczęściej używane polecenia Git

Zestawienie poleceń Git, z których korzystano podczas pracy nad projektem lub które są potrzebne przy rozwiązywaniu problemów, zawiera tabela 3.5.

**Tab. 3.5.** Najczęściej używane polecenia Git

| **Polecenie** | **Przeznaczenie** |
|---|---|
| `git clone` | pobranie repozytorium z GitHuba na komputer |
| `git pull` | pobranie i scalenie zmian z serwera |
| `git checkout -b` | utworzenie nowej gałęzi i przejście na nią |
| `git add` | dodanie plików do poczekalni |
| `git commit -m` | zapisanie zmian w historii wraz z opisem |
| `git push` | wysłanie zmian na GitHub |
| `git status` | sprawdzenie stanu plików |
| `git diff` | podgląd różnic w plikach |
| `git log --oneline` | skrócona historia commitów |
| `git merge` | scalenie gałęzi |
| `git merge --abort` | przerwanie scalania w razie konfliktu |

# 4. Implementacja

## 4.1. Struktura Node i deklaracja klasy BST

Plik nagłówkowy `BST.h` (listing 3) zawiera strukturę `Node` oraz deklarację klasy `BST`. Konstruktor węzła ustawia wartość i zeruje oba wskaźniki. Metody rekurencyjne, które realizują właściwe operacje na węzłach, są prywatne – użytkownik klasy korzysta z publicznych metod, które wywołują je, zaczynając od korzenia. Metody `getRoot` i `setRoot` udostępniają korzeń klasie `FileManager`.

```
#ifndef BST_H
#define BST_H

#include <iostream>
#include <vector>
#include <fstream>
#include <string>

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class BST {
private:
    Node* root;

    Node* insertRec(Node* node, int data);
    Node* deleteRec(Node* node, int data);
    Node* minValueNode(Node* node);
    void destroyTree(Node* node);
    bool searchPathRec(Node* node, int data, std::vector<int>& path
    );
    void saveToTextFileRec(Node* node, std::ofstream& file);

    void printPreorder(Node* node);
    void printInorder(Node* node);
    void printPostorder(Node* node);

public:
    BST();
    ~BST();
```

```
34      void insert(int data);
35      void remove(int data);
36      void clear();
37      void searchPath(int data);
38      void display(int method);
39      void saveToTextFile(const std::string& filename);
40
41      Node* getRoot() const { return root; }
42      void setRoot(Node* newRoot) { root = newRoot; }
43  };
44
45  #endif
```

**Listing 3.** Plik BST.h

## 4.2. Konstruktor i destruktor

Konstruktor (listing 4) tworzy puste drzewo, ustawiając wskaźnik `root` na `nullptr`. Destruktor wywołuje metodę `clear`, dzięki czemu pamięć zajmowana przez wszystkie węzły jest zwalniana automatycznie przy zniszczeniu obiektu drzewa.

```
1  #include "BST.h"
2
3  using namespace std;
4
5  BST::BST() : root(nullptr) {}
6
7  BST::~BST() {
8      clear();
9  }
```

**Listing 4.** Konstruktor i destruktor klasy BST

## 4.3. Dodawanie elementu

Publiczna metoda `insert` (listing 5) przypisuje korzeniowi wynik funkcji rekurencyjnej `insertRec`. Funkcja ta zwraca wskaźnik na korzeń poddrzewa, do którego dodano element. Dzięki temu przypisania w kolejnych wywołaniach rekurencyjnych (`node->left = ...`, `node->right = ...`) automatycznie podłączają nowy węzeł we właściwym miejscu. Gdy funkcja natrafi na `nullptr`, tworzy tam nowy węzeł. Jeśli wartość jest równa wartości napotkanego węzła, żaden z warunków nie jest spełniony i drzewo pozostaje bez zmian.

```
void BST::insert(int data) {
    root = insertRec(root, data);
}

Node* BST::insertRec(Node* node, int data) {
    if (node == nullptr) return new Node(data);
    if (data < node->data) node->left = insertRec(node->left, data)
    ;
    else if (data > node->data) node->right = insertRec(node->right
    , data);
    return node;
}
```

**Listing 5.** Metody insert i insertRec

## 4.4. Usuwanie elementu

Metoda `remove` (listing 6) działa analogicznie do `insert`. Funkcja `deleteRec` odnajduje węzeł o zadanej wartości, a następnie obsługuje trzy opisane w rozdziale 2 przypadki. Dla węzła bez lewego potomka (co obejmuje także liść) zwracany jest wskaźnik na prawego potomka, a węzeł jest zwalniany. Analogicznie postępuje się, gdy brakuje prawego potomka. Jeśli węzeł ma dwóch potomków, funkcja `minValueNode` odnajduje najmniejszą wartość w prawym poddrzewie (idąc stale w lewo). Wartość ta jest kopiowana do usuwanego węzła, a następnie jej pierwotny węzeł jest usuwany z prawego poddrzewa – tam spełnia już warunki jednego z prostszych przypadków. Jeśli szukanej wartości nie ma w drzewie, rekurencja dochodzi do `nullptr` i drzewo nie ulega zmianie.

```
void BST::remove(int data) {
    root = deleteRec(root, data);
}

Node* BST::minValueNode(Node* node) {
    Node* current = node;
    while (current && current->left != nullptr)
        current = current->left;
    return current;
}

Node* BST::deleteRec(Node* node, int data) {
    if (node == nullptr) return node;

```

```
15      if (data < node->data) node->left = deleteRec(node->left, data)
    ;
16      else if (data > node->data) node->right = deleteRec(node->right
    , data);
17      else {
18          if (node->left == nullptr) {
19              Node* temp = node->right;
20              delete node;
21              return temp;
22          } else if (node->right == nullptr) {
23              Node* temp = node->left;
24              delete node;
25              return temp;
26          }
27          Node* temp = minValueNode(node->right);
28          node->data = temp->data;
29          node->right = deleteRec(node->right, temp->data);
30      }
31      return node;
32  }
```

**Listing 6.** Metody remove, minValueNode i deleteRec

## 4.5. Usuwanie całego drzewa

Metoda `clear` (listing 7) wywołuje funkcję `destroyTree`, a następnie zeruje wskaźnik `root`, aby drzewo było poprawnie puste i nadawało się do dalszego użycia. Funkcja `destroyTree` przechodzi drzewo w porządku *postorder*: najpierw zwalnia oba poddrzewa, a dopiero potem sam węzeł. Gdyby kolejność była odwrotna, po zwolnieniu węzła zostałyby utracone wskaźniki na jego potomków.

```
1   void BST::clear() {
2       destroyTree(root);
3       root = nullptr;
4   }
5
6   void BST::destroyTree(Node* node) {
7       if (node != nullptr) {
8           destroyTree(node->left);
9           destroyTree(node->right);
10          delete node;
11      }
12  }
```

**Listing 7.** Metody clear i destroyTree

## 4.6. Wyszukiwanie drogi do elementu

Metoda `searchPath` (listing 8) tworzy pusty wektor `path` i przekazuje go do funkcji `searchPathRec`. Funkcja dopisuje wartość bieżącego węzła do wektora, a następnie sprawdza, czy jest to szukana wartość. Jeśli nie, przeszukuje najpierw lewe, a potem prawe poddrzewo. Gdy żadne z nich nie zawiera szukanej wartości, ostatni element jest usuwany z wektora (`pop_back`) i funkcja zwraca `false`. Po zakończeniu poszukiwań wektor zawiera dokładnie wartości węzłów na drodze od korzenia do szukanego elementu. Metoda wypisuje je, rozdzielając strzałkami, albo informuje, że elementu nie znaleziono.

```
void BST::searchPath(int data) {
    vector<int> path;
    if (searchPathRec(root, data, path)) {
        cout << "Droga do " << data << ": ";
        for (int val : path) cout << val << " -> ";
        cout << "Koniec\n";
    } else {
        cout << "Element nie znaleziony.\n";
    }
}

bool BST::searchPathRec(Node* node, int data, vector<int>& path) {
    if (node == nullptr) return false;
    path.push_back(node->data);
    if (node->data == data) return true;
    if (searchPathRec(node->left, data, path) || searchPathRec(node
   ->right, data, path))
        return true;
    path.pop_back();
    return false;
}
```

**Listing 8.** Metody searchPath i searchPathRec

## 4.7. Wyświetlanie drzewa

Metoda `display` (listing 9) przyjmuje numer metody przechodzenia: 1 – *preorder*, 2 – *inorder*, 3 – *postorder*. Dla pustego drzewa wypisuje komunikat i kończy działanie, a dla nieznanego numeru metody – komunikat o błędzie. Trzy prywatne funkcje przechodzenia różnią się wyłącznie położeniem instrukcji wypisującej wartość węzła względem wywołań rekurencyjnych, co odpowiada tabeli 2.1.

```cpp
void BST::display(int method) {
    if (root == nullptr) {
        cout << "Drzewo jest puste.\n";
        return;
    }
    switch (method) {
        case 1: printPreorder(root); break;
        case 2: printInorder(root); break;
        case 3: printPostorder(root); break;
        default: cout << "Nieznana metoda.\n";
    }
    cout << endl;
}

void BST::printPreorder(Node* node) {
    if (node == nullptr) return;
    cout << node->data << " ";
    printPreorder(node->left);
    printPreorder(node->right);
}

void BST::printInorder(Node* node) {
    if (node == nullptr) return;
    printInorder(node->left);
    cout << node->data << " ";
    printInorder(node->right);
}

void BST::printPostorder(Node* node) {
    if (node == nullptr) return;
    printPostorder(node->left);
    printPostorder(node->right);
    cout << node->data << " ";
}
```

**Listing 9.** Metody wyświetlania drzewa

## 4.8. Zapis drzewa do pliku tekstowego

Metoda `saveToTextFile` (listing 10) otwiera plik o podanej nazwie do zapisu (istniejący plik zostaje nadpisany). Jeśli otwarcie się powiodło, funkcja `saveToTextFileRec` zapisuje wartości węzłów w kolejności *preorder*, każdą z dodaną spacją, po czym plik jest zamykany i wypisywany jest komunikat o powodzeniu. W przeciwnym razie wypisywana jest informacja o błędzie.

```
void BST::saveToTextFile(const string& filename) {
    ofstream file(filename);
    if (file.is_open()) {
        saveToTextFileRec(root, file);
        file.close();
        cout << "Zapisano do pliku tekstowego.\n";
    } else {
        cout << "Nie mozna otworzyc pliku.\n";
    }
}

void BST::saveToTextFileRec(Node* node, ofstream& file) {
    if (node == nullptr) return;
    file << node->data << " ";
    saveToTextFileRec(node->left, file);
    saveToTextFileRec(node->right, file);
}
```

**Listing 10.** Metody saveToTextFile i saveToTextFileRec

## 4.9. Klasa FileManager

Klasa `FileManager` (listing 11) zawiera trzy publiczne metody operujące na obiekcie drzewa przekazanym przez referencję oraz dwie prywatne funkcje rekurencyjne odpowiedzialne za zapis i odczyt struktury drzewa w formacie binarnym.

```
#ifndef FILEMANAGER_H
#define FILEMANAGER_H

#include "BST.h"
#include <string>

class FileManager {
private:
    void saveBinaryRec(Node* node, std::ofstream& file);
    Node* loadBinaryRec(std::ifstream& file);

public:
    void saveBinary(BST& tree, const std::string& filename);
    void loadBinary(BST& tree, const std::string& filename);
    void loadFromText(BST& tree, const std::string& filename);
};

```

```
#endif
```

**Listing 11.** Plik FileManager.h

### 4.9.1. Zapis binarny

Metoda `saveBinary` (listing 12) otwiera plik w trybie binarnym i uruchamia funkcję `saveBinaryRec` od korzenia drzewa. Funkcja zapisuje wartość każdego węzła jako liczbę typu `int` za pomocą metody `write`, a w miejscu pustego wskaźnika zapisuje znacznik `-1`. Przejście odbywa się w porządku *preorder*, co daje układ danych opisany w tabeli 2.3.

```
void FileManager::saveBinary(BST& tree, const string& filename) {
    ofstream file(filename, ios::binary);
    if (file.is_open()) {
        saveBinaryRec(tree.getRoot(), file);
        file.close();
        cout << "Zapisano binarnie.\n";
    } else {
        cout << "Blad zapisu pliku binarnego.\n";
    }
}

void FileManager::saveBinaryRec(Node* node, ofstream& file) {
    int marker = -1;
    if (node == nullptr) {
        file.write(reinterpret_cast<char*>(&marker), sizeof(int));
        return;
    }
    file.write(reinterpret_cast<char*>(&node->data), sizeof(int));
    saveBinaryRec(node->left, file);
    saveBinaryRec(node->right, file);
}
```

**Listing 12.** Metody saveBinary i saveBinaryRec

### 4.9.2. Odczyt binarny

Metoda `loadBinary` (listing 13) po pomyślnym otwarciu pliku najpierw usuwa dotychczasową zawartość drzewa (`clear`), a następnie ustawia korzeń na drzewo zbudowane przez `loadBinaryRec`. Funkcja ta odczytuje kolejną liczbę z pliku. Jeśli odczyt się nie powiódł (koniec pliku) lub odczytano znacznik `-1`, zwraca `nullptr`. W przeciwnym razie tworzy węzeł z odczytaną wartością i rekurencyjnie odtwarza

jego lewe, a potem prawe poddrzewo. Kolejność odczytu jest taka sama jak kolejność zapisu, dzięki czemu odtwarzana jest dokładnie ta sama struktura. Jeśli plik nie może zostać otwarty, dotychczasowe drzewo pozostaje nienaruszone.

```
void FileManager::loadBinary(BST& tree, const string& filename) {
    ifstream file(filename, ios::binary);
    if (file.is_open()) {
        tree.clear();
        tree.setRoot(loadBinaryRec(file));
        file.close();
        cout << "Wczytano binarnie.\n";
    } else {
        cout << "Blad odczytu pliku binarnego.\n";
    }
}

Node* FileManager::loadBinaryRec(ifstream& file) {
    int data;
    if (!file.read(reinterpret_cast<char*>(&data), sizeof(int)) ||
   data == -1) {
        return nullptr;
    }
    Node* node = new Node(data);
    node->left = loadBinaryRec(file);
    node->right = loadBinaryRec(file);
    return node;
}
```

**Listing 13.** Metody loadBinary i loadBinaryRec

### 4.9.3. Odczyt z pliku tekstowego

Metoda `loadFromText` (listing 14) wczytuje z pliku kolejne liczby całkowite i dodaje każdą do drzewa metodą `insert`. Pętla kończy się, gdy odczyt kolejnej liczby się nie powiedzie, czyli po dojściu do końca pliku lub napotkaniu znaków niebędących liczbą. W odróżnieniu od odczytu binarnego metoda **nie czyści** drzewa przed wczytaniem – liczby z pliku są dodawane do istniejącej zawartości.

```
void FileManager::loadFromText(BST& tree, const string& filename) {
    ifstream file(filename);
    if (file.is_open()) {
        int val;
        while (file >> val) {
            tree.insert(val);
        }
```

```
        file.close();
        cout << "Wczytano liczby z pliku tekstowego.\n";
    } else {
        cout << "Nie mozna otworzyc pliku tekstowego.\n";
    }
}
```

**Listing 14.** Metoda loadFromText

## 4.10. Program główny

Funkcja `main` (listing 15) tworzy obiekt drzewa oraz obiekt klasy `FileManager`, a następnie w pętli `do...while` wyświetla menu (funkcja `printMenu`) i wykonuje wybraną operację za pomocą instrukcji `switch`. Pętla kończy się, gdy użytkownik wybierze opcję 0. Po zakończeniu programu destruktor obiektu `tree` zwalnia pamięć zajmowaną przez wszystkie węzły.

```
#include "BST.h"
#include "FileManager.h"
#include <iostream>
#include <string>

using namespace std;

void printMenu() {
    cout << "\n--- MENU ---\n";
    cout << "1. Dodaj element\n";
    cout << "2. Usun element z drzewa\n";
    cout << "3. Usun cale drzewo\n";
    cout << "4. Szukaj drogi do elementu\n";
    cout << "5. Wyswietl drzewo\n";
    cout << "6. Zapisz drzewo do pliku TXT\n";
    cout << "7. Zapisz drzewo binarnie\n";
    cout << "8. Wczytaj drzewo binarnie\n";
    cout << "9. Wczytaj liczby z pliku TXT\n";
    cout << "0. Wyjscie\n";
    cout << "Wybierz opcje: ";
}

int main() {
    BST tree;
    FileManager fileManager;
    int choice, val;
    string filename;
```

```
28
29      do {
30          printMenu();
31          cin >> choice;
32
33          switch(choice) {
34              case 1:
35                  cout << "Podaj wartosc: "; cin >> val;
36                  tree.insert(val);
37                  break;
38              case 2:
39                  cout << "Podaj wartosc do usuniecia: "; cin >> val;
40                  tree.remove(val);
41                  break;
42              case 3:
43                  tree.clear();
44                  cout << "Drzewo usuniete.\n";
45                  break;
46              case 4:
47                  cout << "Podaj wartosc do znalezienia: "; cin >>
    val;
48                  tree.searchPath(val);
49                  break;
50              case 5:
51                  int method;
52                  cout << "Metoda (1-pre, 2-in, 3-post): "; cin >>
    method;
53                  tree.display(method);
54                  break;
55              case 6:
56                  cout << "Podaj nazwe pliku TXT do zapisu (np.
    drzewo.txt): "; cin >> filename;
57                  tree.saveToTextFile(filename);
58                  break;
59              case 7:
60                  cout << "Podaj nazwe pliku binarnego do zapisu (np.
     drzewo.bin): "; cin >> filename;
61                  fileManager.saveBinary(tree, filename);
62                  break;
63              case 8:
64                  cout << "Podaj nazwe pliku binarnego do odczytu: ";
     cin >> filename;
65                  fileManager.loadBinary(tree, filename);
66                  break;
67              case 9:
```

```
68                 cout << "Podaj nazwe pliku TXT z liczbami: "; cin
   >> filename;
69                 fileManager.loadFromText(tree, filename);
70                 break;
71             }
72         } while (choice != 0);
73
74         return 0;
75 }
```

**Listing 15.** Plik main.cpp

## 4.11. Przykładowy przebieg programu

Poniżej opisano scenariusz użycia programu na drzewie z rysunku 2.1. Najpierw opcją 1 dodano kolejno wartości 50, 30, 70, 20, 40, 60 i 80. Następnie wykonano operacje przedstawione na listingu 16 (dla czytelności pominięto powtarzające się menu, a po znaku : podano wartości wpisane przez użytkownika).

```
Opcja 5, metoda 1:   50 30 20 40 70 60 80
Opcja 5, metoda 2:   20 30 40 50 60 70 80
Opcja 5, metoda 3:   20 40 30 60 80 70 50
Opcja 4, wartosc 60: Droga do 60: 50 -> 70 -> 60 -> Koniec
Opcja 4, wartosc 99: Element nie znaleziony.
Opcja 2, wartosc 70: (usuniecie elementu, brak komunikatu)
Opcja 5, metoda 2:   20 30 40 50 60 80
Opcja 6, drzewo.txt: Zapisano do pliku tekstowego.
Opcja 7, drzewo.bin: Zapisano binarnie.
Opcja 3:             Drzewo usuniete.
Opcja 5, metoda 2:   Drzewo jest puste.
Opcja 8, drzewo.bin: Wczytano binarnie.
Opcja 5, metoda 2:   20 30 40 50 60 80
```

**Listing 16.** Przykładowy przebieg programu

Wyniki odpowiadają wartościom z tabeli 2.1. Po usunięciu wartości 70 (węzeł z dwoma potomkami) jej miejsce zajmuje następnik 80, co odpowiada rysunkowi 2.2. Odczyt pliku binarnego po wyczyszczeniu drzewa przywraca zapisaną strukturę.

## 4.12. Historia projektu w repozytorium

Przebieg prac w serwisie GitHub przedstawia tabela 4.1. Commity Patryka pochodzą z konta `pgarhub`, a Damiana z kont `Damian701` i `Damian`. Wiersze zaczynające się od *Merge pull request* to scalenia gałęzi z gałęzią główną.

**Tab. 4.1.** Historia commitów projektu

| Data | Skrót | Autor | Opis |
|---|---|---|---|
| 29.10.2026 | `d251959` | Patryk | Dodaj .gitattributes i .gitignore. |
| 29.10.2026 | `b7eeaf6` | Patryk | Dodanie struktury plików projektu |
| 02.11.2026 | `3644ff9` | Damian | Dodanie struktury Node i deklaracji BST |
| 02.11.2026 | `527f4f2` | Damian | Implementacja dodawania elementow do drzewa |
| 02.11.2026 | `41fb84d` | Damian | Dodanie glownego menu i test dodawania |
| 02.11.2026 | `c935c96` | Damian | Merge pull request #1 from pgarhub/galaz-damiana |
| 02.11.2026 | `2f6dac2` | Patryk | Deklaracje metod wyswietlania drzewa |
| 02.11.2026 | `718a52c` | Patryk | Implementacja wyswietlania (preorder, inorder, postorder) |
| 02.11.2026 | `5c8fc35` | Patryk | Dodanie opcji wyswietlania do menu |
| 02.11.2026 | `5de03d0` | Patryk | Merge pull request #2 from pgarhub/galaz-patryka |
| 03.11.2026 | `be19406` | Damian | Dodanie dokumentacji |
| 06.11.2026 | `52d6691` | Damian | Dodanie czyszczenia drzewa i szukania drogi |
| 06.11.2026 | `058ffe8` | Damian | Merge pull request #3 from pgarhub/damian-etap2 |
| 06.11.2026 | `24f9f25` | Damian | Dodanie READme |
| 09.11.2026 | `58bb204` | Patryk | Dodanie menadzera plikow i obslugi zapisu/odczytu |
| 09.11.2026 | `0792984` | Patryk | Merge pull request #4 from pgarhub/patryk-etap2 |

## 4.13. Podsumowanie aktywności w repozytorium

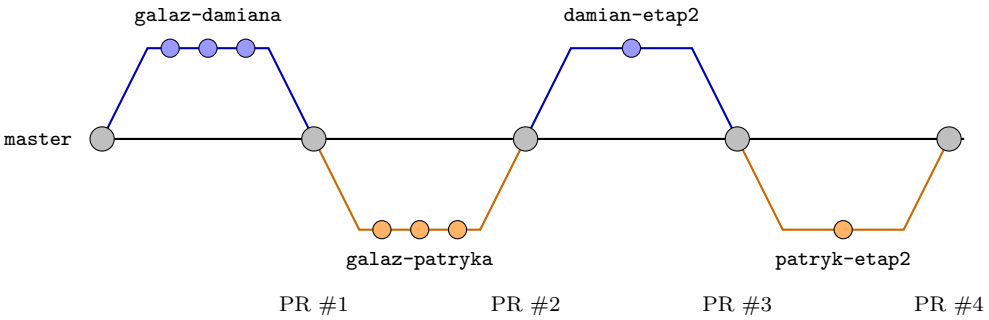
Praca nad projektem trwała od 29 października do 9 listopada 2026 roku (12 dni), a commity zapisywano w pięciu dniach: 29.10, 02.11, 03.11, 06.11 i 09.11. W historii znajduje się 16 wpisów: 12 zwykłych commitów i 4 scalenia *pull requestów*. Wkład obu autorów był zbliżony (tabela 4.2).

Uproszczony schemat gałęzi i scaleń przedstawia rysunek 4.1. Każda gałąź robo-

**Tab. 4.2.** Aktywność autorów w repozytorium

| Miara | Damian | Patryk |
|---|---|---|
| Commity (bez scaleń) | 6 | 6 |
| Wykonane scalenia *pull requestów* | 2 | 2 |
| Dni, w których zapisano commity | 3 | 3 |

cza powstawała z aktualnej gałęzi głównej i była do niej scalana przez *pull request*, dzięki czemu historia ma liniowy, czytelny przebieg.




**Rys. 4.1.** Uproszczony schemat gałęzi i scaleń (niebieskie punkty – commity Damiana, pomarańczowe – Patryka, szare – scalenia na gałęzi `master`; pominięto commity dokumentacji i pliku `README`)

## 4.14. Konflikty scalania w projekcie

W trakcie realizacji projektu **nie wystąpił żaden konflikt scalania**. Wszystkie cztery *pull requesty* (tabela 3.4) zostały scalone z gałęzią główną automatycznie, bez ręcznej ingerencji w pliki.

Do uniknięcia konfliktów przyczyniło się kilka zasad przyjętych w zespole:

- **osobne gałęzie** – każdy autor pracował na własnej gałęzi i nie zapisywał zmian bezpośrednio w gałęzi głównej,
- **podział zadań** – zadania były podzielone tematycznie (tabela 3.3): jeden autor zajmował się dodawaniem, czyszczeniem i szukaniem drogi, drugi wyświetlaniem oraz zapisem i odczytem z plików,
- **małe commity** – każdy commit obejmował jedną niewielką zmianę, dzięki czemu zakres zmian w plikach był niewielki,

- **podział na etapy** – zmiany z jednego etapu były scalane przed rozpoczęciem kolejnego,
- **osobna klasa** – obsługę plików umieszczono w klasie `FileManager`, w nowych plikach, które nie były edytowane przez drugiego autora.

Mimo że konflikty się nie pojawiły, należy je brać pod uwagę przy pracy zespołowej. Największe ryzyko występuje w plikach, do których obaj autorzy dopisują własne fragmenty: w deklaracji klasy `BST` (`BST.h`), w pliku `BST.cpp` oraz w funkcji `main` (menu i instrukcja `switch`). Gdyby konflikt wystąpił, należałoby postąpić według procedury z listingu 17.

```
git checkout nazwa-galezi              # przejscie na wlasna galaz
git merge master                       # scalanie zmian z galezi
   glownej
git status                             # lista plikow z konfliktem
# reczna edycja pliku: usuniecie znacznikow, zachowanie obu wersji
git add nazwa-pliku                    # oznaczenie pliku jako
   rozwiazanego
git commit                             # zapis wyniku scalenia
git push                               # wyslanie rozwiazanej galezi
git merge --abort                      # przerwanie scalania (w
   razie potrzeby)
```

**Listing 17.** Procedura rozwiązania konfliktu scalania

# 5. Wnioski

W ramach projektu zaimplementowano w języku C++ binarne drzewo poszukiwań alokowane na stercie. Zrealizowano wszystkie założone operacje: dodawanie i usuwanie elementów, usuwanie całego drzewa, wyszukiwanie drogi do elementu, wyświetlanie drzewa w porządkach *preorder*, *inorder* i *postorder* oraz zapis i odczyt drzewa z plików tekstowych i binarnych. Całość obsługiwana jest z poziomu menu w konsoli.

Implementacja potwierdziła, że rekurencja jest naturalnym sposobem zapisu operacji na drzewie – większość metod ma zwięzłą postać, a ich poprawność łatwo uzasadnić na podstawie własności drzewa BST. Najbardziej złożony okazał się przypadek usuwania węzła z dwoma potomkami, w którym trzeba zachować porządek wartości przez zastąpienie usuwanego elementu jego następnikiem. Zastosowanie przypisań wartości zwracanych przez funkcje rekurencyjne (`node->left = ...`) pozwoliło uniknąć ręcznej obsługi wskaźnika na rodzica. Rozdzielenie obsługi plików na osobną klasę `FileManager` oddzieliło logikę drzewa od operacji wejścia-wyjścia. Zapis *preorder* okazał się wygodny w obu formatach: w tekstowym pozwala odtworzyć kształt drzewa przez zwykłe dodawanie elementów, a w binarnym – dzięki znacznikom pustych węzłów – odtworzyć strukturę bez dodatkowych porównań.

Podczas analizy zauważono ograniczenia obecnej wersji, które można usunąć w dalszych pracach:

- w formacie binarnym wartość `-1` jest znacznikiem pustego węzła, więc nie można zapisać w nim węzła o takiej wartości; po wczytaniu pliku zawierającego taki węzeł drzewo zostałoby obcięte w tym miejscu,
- metoda `searchPath` przeszukuje oba poddrzewa każdego węzła i nie wykorzystuje własności drzewa BST; porównanie szukanej wartości z wartością węzła pozwoliłoby ograniczyć się do jednej gałęzi i uzyskać złożoność $O(h)$,
- program nie waliduje danych wprowadzanych z klawiatury, a odczyt z pliku binarnego nie sprawdza, czy wczytana struktura spełnia własność drzewa BST,
- próba dodania istniejącej wartości oraz usunięcia nieistniejącej jest cicho ignorowana; lepsza byłaby sygnalizacja takiej sytuacji użytkownikowi,
- drzewo nie jest równoważone, więc dodawanie wartości w kolejności rosnącej lub malejącej tworzy drzewo zdegenerowane o złożoności operacji $O(n)$ i dużej głębokości rekurencji,

- klasa `BST` zarządza pamięcią ręcznie, ale nie definiuje konstruktora kopiującego ani operatora przypisania; skopiowanie obiektu drzewa prowadziłoby do dwukrotnego zwolnienia tych samych węzłów,
- drzewo przechowuje wyłącznie wartości typu `int`; uogólnienie można uzyskać przez zastosowanie szablonów (`template`), a zamiast surowych wskaźników można użyć inteligentnych wskaźników z biblioteki standardowej.

Praca w zespole z użyciem Gita i GitHuba wymusiła podział zadań na etapy i osobne gałęzie, a historia commitów pozwala prześledzić wkład każdego z autorów. W trakcie projektu nie wystąpiły konflikty scalania, co wynikało z pracy na osobnych gałęziach, tematycznego podziału zadań i niewielkich commitów. Największe ryzyko konfliktu dotyczy plików, do których dopisują obaj autorzy (`BST.h`, `BST.cpp`, `main.cpp`), dlatego warto pobierać zmiany z gałęzi głównej przed wysłaniem własnych i ustalać, kto odpowiada za który fragment plików wspólnych.

Użycie serwisu GitHub dało w projekcie kilka wymiernych korzyści: kopię zapasową kodu poza komputerami autorów, pełną historię zmian z możliwością powrotu do wcześniejszej wersji, równoległą pracę dwóch osób na osobnych gałęziach oraz przegląd zmian przed ich włączeniem dzięki pull requestom. Czytelne opisy commitów i numerowane pull requesty sprawiły, że przebieg prac można odtworzyć na podstawie samej historii repozytorium, a podział zadań między autorami jest w niej jednoznacznie widoczny.

Mimo wskazanych ograniczeń program poprawnie realizuje wszystkie wymagane funkcje, a przeprowadzony scenariusz testowy (listing 16) potwierdza zgodność wyników z oczekiwaniami wynikającymi z analizy problemu.

# Literatura

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Wprowadzenie do algorytmów*, Wydawnictwo Naukowe PWN, Warszawa 2012.

[2] B. Stroustrup, *Język C++. Kompendium wiedzy*, Helion, Gliwice 2014.

[3] *cppreference.com*, strona internetowa, https://en.cppreference.com.

[4] S. Chacon, B. Straub, *Pro Git*, wyd. 2, Apress 2014, https://git-scm.com/book.

[5] *GitHub Docs*, strona internetowa, https://docs.github.com.

# Spis rysunków

2.1. Przykładowe drzewo BST utworzone przez dodanie wartości 50, 30, 70, 20, 40, 60, 80 . . . . . 7

2.2. Usunięcie węzła z dwoma potomkami: wartość 70 zostaje zastąpiona następnikiem (80) . . . . . 8

3.1. Model klas programu . . . . . 16

3.2. Zrzut ekranu terminala: utworzenie gałęzi, commit i wysłanie zmian na GitHub . . . . . 18

4.1. Uproszczony schemat gałęzi i scaleń (niebieskie punkty – commity Damiana, pomarańczowe – Patryka, szare – scalenia na gałęzi `master`; pominięto commity dokumentacji i pliku `README`) . . . . . 34

# Spis tabel

1.1. Wymagania funkcjonalne . . . . . 5
2.1. Porządki przechodzenia . . . . . 9
2.2. Złożoność operacji . . . . . 10
2.3. Zawartość pliku binarnego . . . . . 11
2.4. Elementy repozytorium . . . . . 13
3.1. Pliki projektu . . . . . 15
3.2. Opcje menu . . . . . 17
3.3. Podział zadań . . . . . 17
3.4. Gałęzie i pull requesty . . . . . 18
3.5. Polecenia Git . . . . . 20
4.1. Historia commitów . . . . . 33
4.2. Aktywność autorów . . . . . 34

# Spis listingów

1. Polecenia Git użyte podczas commitowania . . . . . 18
2. Podstawowe polecenia konfiguracji i przeglądania repozytorium . . . . 19
3. Plik BST.h . . . . . 21
4. Konstruktor i destruktor klasy BST . . . . . 22
5. Metody insert i insertRec . . . . . 23
6. Metody remove, minValueNode i deleteRec . . . . . 23
7. Metody clear i destroyTree . . . . . 24
8. Metody searchPath i searchPathRec . . . . . 25
9. Metody wyświetlania drzewa . . . . . 25
10. Metody saveToTextFile i saveToTextFileRec . . . . . 27
11. Plik FileManager.h . . . . . 27
12. Metody saveBinary i saveBinaryRec . . . . . 28
13. Metody loadBinary i loadBinaryRec . . . . . 29
14. Metoda loadFromText . . . . . 29
15. Plik main.cpp . . . . . 30
16. Przykładowy przebieg programu . . . . . 32
17. Procedura rozwiązania konfliktu scalania . . . . . 35